CONTENTS

7. Instructor pre-flight (the week before)

Lab Setup — the isolated security lab

The single source of truth for the teaching lab. Serves both the 20-day course (Days 12–20) and the SecureCorp 4-week capstone. Read §0 before anything else.

Verified against: VirtualBox 7.0/7.1 · Kali 2025.x · Metasploitable2 (SourceForge/Rapid7) · DVWA v1.10 · OWASP Juice Shop 17.x · Wazuh 4.9. Re-check tool versions before a new cohort — installer commands change between releases.

---

0. Principles — non-negotiable

1. **The lab is isolated.** The vulnerable machines (Metasploitable2, DVWA, Juice Shop) must **never** be reachable from a network with devices you don't own — not your home Wi-Fi, not the campus network, not a phone on the same LAN. In VirtualBox that means an **Internal Network** or **Host-Only** adapter — **never Bridged** for a target. *Why:* Metasploitable2 ships a root backdoor on port 1524, a backdoored vsftpd, an open distcc RCE, world-readable NFS. On a real network it is a free foothold for an actual attacker.
2. **Snapshot before you break things.** Every attack day starts from a clean snapshot.
3. **Everything you do produces evidence** — a screenshot, terminal output, or a log excerpt.
4. **Scope = your lab only.** The only IPs you ever scan or attack are the lab targets you were assigned. Never a broad range, never a real host. (Course Day 1; capstone ROE.)

---

1. Two models — pick one per programme

	Model A — instructor-hosted central lab	**Model B — student-built isolated lab**
Used by	**20-day course, Days 12–20**	**SecureCorp 4-week capstone** (building it *is* Task 1.1)
Targets	one server the instructor runs (DVWA, Juice Shop, Metasploitable2, a vuln-LLM)	each student runs their own target VM(s)
Student's machine	a light **Kali VM**, or an account on the **shared class Kali**	a Kali/Ubuntu **attacker VM they build**
Network	class LAN or a VPN to the target server; targets never on the public internet	**VirtualBox Internal Network**, fully offline
RAM per student	4–8 GB (thin client)	8 GB (manual-log path) / 16 GB (with Wazuh)
Best for	large cohorts, weak laptops, a fast start	teaching *how a lab is built and isolated* as a graded skill

You can run both — the target **images are identical**; only who hosts them changes.

---

2. Model A — instructor-hosted central lab (20-day course)

2.1 THE TARGET HOST

One machine the instructor controls: a spare desktop, a mini-PC, or a small cloud VM. **8 GB RAM, 60 GB disk, 4 vCPU** is comfortable for all targets at once.

Install Docker/Podman + Compose, then:

```
# targets/compose.yml  —  run:  docker compose up -d
services:
  dvwa:
    image: ghcr.io/digininja/dvwa:latest
    ports: ["8080:80"]
    restart: unless-stopped
  juiceshop:
    image: bkimminich/juice-shop:latest
    ports: ["3000:3000"]
    restart: unless-stopped
  vulnllm:
    build: ./vulnllm          # the Day-18 fallback app (see §5)
    ports: ["8888:8888"]
    restart: unless-stopped
```

Metasploitable2 does not containerise cleanly — run it as a VM **on the same host** (VirtualBox headless) on a host-only/internal adapter, and expose only the ports you need to the class subnet with a firewall rule, **or** use the community image `tleemcjr/metasploitable2` if your exercises don't need the kernel-level vulns.

2.2 LOCK THE TARGET HOST DOWN AFTER SETUP

1. Pull all images / boot all VMs **once** with internet.
2. Then remove the host's default route, or firewall it:

```
# allow only the class subnet in; drop everything outbound except DNS/NTP
iptables -P INPUT DROP
iptables -A INPUT -s <CLASS_SUBNET>/24 -j ACCEPT
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -P OUTPUT DROP
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

3. Metasploitable2 specifically: give it **only** a host-only/internal adapter — no NAT, no bridge — and reach it from the class Kali which straddles both networks.

2.3 HOW STUDENTS CONNECT (PICK ONE)

- **Same physical LAN** — a classroom switch; students' Kali VMs on a bridged adapter *to the classroom switch only* (which has no uplink), target host on the same switch. Simplest when you have the room.
- **WireGuard VPN** — one `.conf` per student; the target host is the only thing on the VPN subnet. Works for remote/hybrid. Give students `10.13.13.x`; target host `10.13.13.1`.
- **SSH to a shared Kali** — one Linux account per student on a Kali box that sits on the target subnet. The weak-laptop path: a Chromebook + `ssh` + a browser is enough.

2.4 PER-STUDENT ATTACK BOX

Provide **one Kali `.ova`** (build it once, export, distribute):

- Adapter 1: **NAT** (for `apt`/tool updates; students disable it during exercises if you want strict isolation)
- Adapter 2: **Internal Network** `labnet` or the classroom-switch bridge — static IP
- Pre-installed: `nmap`, `wireshark`, `metasploit-framework`, `hydra`, `john`, `hashcat`, `seclists`, `gobuster`, `nikto`, `curl`, `python3`, `pipx`, Burp Suite Community, `searchsploit`
- `scanner.py` from Day 9 in `~/`
- Students run **VM → Snapshot → "day12-clean"** on Day 12.

Shared-Kali fallback: same tools, one account per student, `/home/<student>`, they `git push` their repo from there.

2.5 THE DAY 18 VULN-LLM

Two options:

- **External:** point students at `gandalf.lakera.ai` (needs internet on the *student* box, not the lab). Simplest.
- **Local:** the `vulnllm` container in §2.1 — a tiny Flask app with a system prompt holding a secret and no input/output filtering. Build file in §5. Use this if the class network blocks Lakera or you want it fully offline.

2.6 VERIFICATION (MODEL A) — EVERY STUDENT, ON DAY 12

```
☐  Kali boots; `ip a` shows an address on the lab adapter
☐  `ping <TARGET_HOST>`  (or `curl -I http://<TARGET_HOST>:8080` if ICMP is
filtered)
☐  http://<TARGET_HOST>:8080  (DVWA)  and  :3000  (Juice Shop)  load in the
Kali browser
☐  `python3 ~/scanner.py <TARGET_HOST>`  lists open ports
☐  `nmap -sV <METASPLOITABLE_IP>`  returns services
☐  VM snapshot "day12-clean" taken
☐  target details recorded in the Engagement Journal (Phase 1)
```

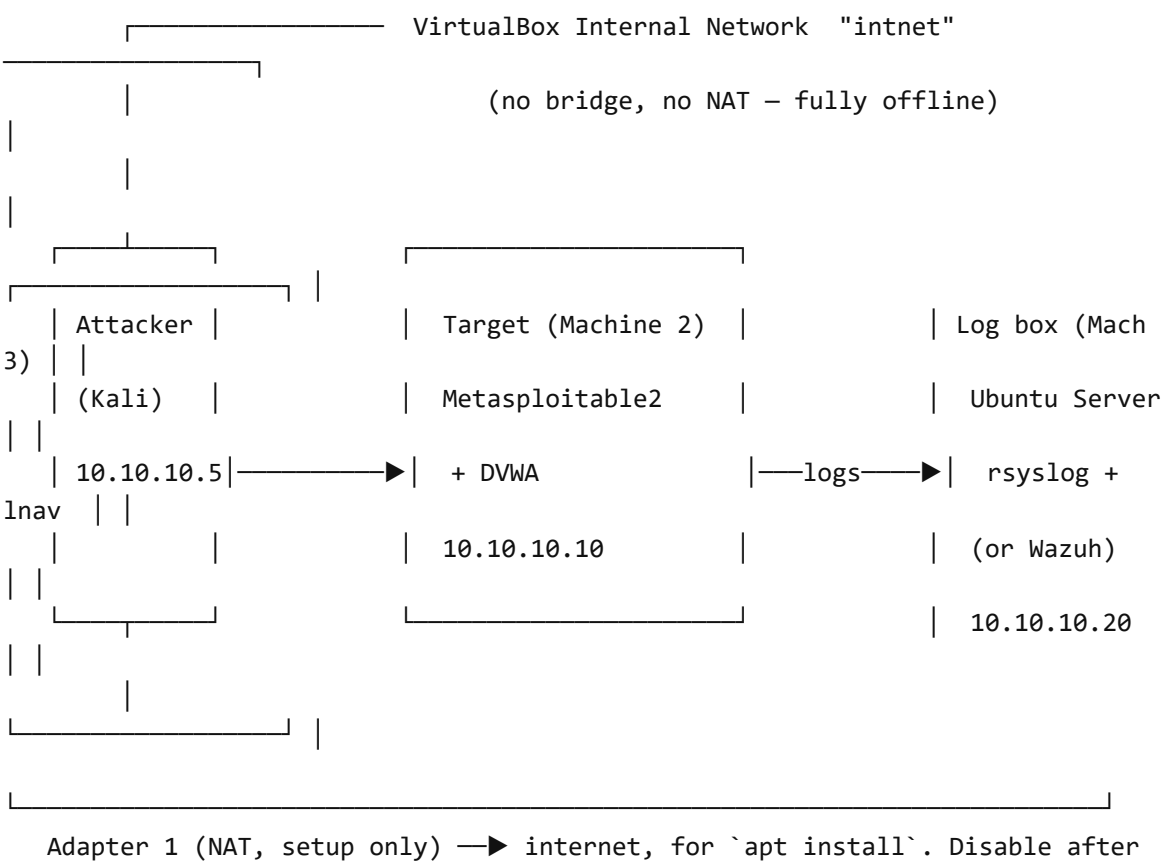
Anyone failing → shared Kali for the day, own VM fixed in office hours. **Nobody starts Day 13 without lab access.**

---

3. Model B — student-built isolated lab (SecureCorp capstone)

Building this is a graded learning objective (Task 1.1) — so students build it, but **safely**. The whole lab lives on one **VirtualBox Internal Network** and never touches a real network.

3.1 ARCHITECTURE



3.2 ATTACKER VM (KALI OR UBUNTU + TOOLS)

- **Adapter 1: NAT** — enabled only while installing tools (`nmap hydra john hashcat metasploit-framework seclists wireshark`).
- **Adapter 2: Internal Network → name it `intnet`** — this is the lab side.
- Static IP on Adapter 2:

```
# /etc/network/interfaces.d/intnet   (Kali/Debian)
auto eth1
iface eth1 inet static
    address 10.10.10.5
    netmask 255.255.255.0
```

- Once tools are installed: **remove or disable Adapter 1** for full isolation (re-enable briefly only when you deliberately need to update something, then disable again).

3.3 TARGET VM — METASPLOITABLE2 + DVWA

Use the pre-built OVA the instructor provides (`securecorp-target.ova`) — Metasploitable2 with DVWA already installed, MySQL configured, and the seed-incident script staged. Building DVWA from scratch on Metasploitable2's ancient PHP stack is a rabbit hole; skip it.

- **One adapter only: Internal Network `intnet`. Never** give this VM NAT or Bridged.
- Static IP (Metasploitable2 runs old Ubuntu — `/etc/network/interfaces`):

  ```
  auto eth0
  iface eth0 inet static
      address 10.10.10.10
      netmask 255.255.255.0
  ```

  then `sudo /etc/init.d/networking restart`.
- Default logins (research changing them is Task 1.1): `msfadmin:msfadmin`. Other accounts that exist for the brute-force tasks: `user:user`, `service:service`, `postgres:postgres`, `sys:batman`. The account you brute-force over SSH is one **you create** with a deliberately weak password.
- DVWA: browse to `http://10.10.10.10/dvwa` from the attacker, run **Setup** → **Create/Reset Database** once, set security to **low** (bump to **medium** for the Challenge tasks).

If there's no pre-built OVA: install DVWA while the target briefly has a NAT adapter — `snapshot → NAT on → install DVWA → NAT off → snapshot "clean"` — then it's isolated forever after.

3.4 LOG BOX — MACHINE 3 (CHOOSE PER HARDWARE)

Path	RAM	What it is	Who
Manual (Core — everyone does this first)	+1 GB	Ubuntu Server + `rsyslog` receiving the target's `auth.log` and Apache log; investigate with `lnav`, `grep`, `journalctl`	all students
Wazuh single-node (Core if you can)	+4–5 GB	Wazuh manager + indexer + dashboard via the official quickstart	students with **16 GB** total
Shared Wazuh	0	connect the target's Wazuh agent to an instructor-run Wazuh server	8 GB students who want the SIEM experience

- **Everyone does the manual path first**, as its own graded step — read the raw `auth.log`, correlate failed logins by source IP and timestamp *by hand*. That is the SOC skill.
- **Then** (Core-if-16GB / Challenge otherwise) deploy Wazuh, ingest the same logs, and answer: *what did the SIEM catch that my manual search missed? what did I catch that it didn't?*
- Custom Wazuh rules that auto-detect one of the web attacks = **Stretch**.

Log box adapter: **Internal Network** `intnet` only. Static IP `10.10.10.20` (netplan on modern Ubuntu):

```
# /etc/netplan/01-intnet.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      addresses: [10.10.10.20/24]
```

`sudo netplan apply`.

rsyslog receiver (manual path) — on the log box:

```
# /etc/rsyslog.d/10-lab.conf
module(load="imudp") input(type="imudp" port="514")
module(load="imtcp") input(type="imtcp" port="514")
$template LabFmt,"/var/log/lab/%HOSTNAME%/%PROGRAMNAME%.log"
*.* ?LabFmt
```

On the target, forward: add `*.* @10.10.10.20:514` (UDP) or `@@` (TCP) to `/etc/rsyslog.conf` (Metasploitable2 uses `sysklogd` — edit `/etc/syslog.conf`: `*.* @10.10.10.20`) and restart the logger. For the Apache/DVWA access log, run a `tail -F ... | logger -n 10.10.10.20` shim, or (cleaner) rsyslog's `imfile` on the target.

3.5 THE SEEDED INCIDENT (FIXES "WEEK 3 HAS NOTHING TO INVESTIGATE")

Before Week 3, the instructor runs **`seed-incident.sh`** on each student's target (or hands out a per-student "collected evidence" bundle). It plants activity the student did **not** do:

- a **successful** SSH login from `10.10.10.66` (an IP not in their notes) at ~03:xx
- a `wget` of `http://10.10.10.66/update.sh` into `/tmp/.cache/`
- a new cron entry in `/etc/cron.d/` running that file every 10 min
- ~30 outbound "beacon" lines to `10.10.10.66:8443`, small and evenly spaced

The script randomises the hour, the filename, and one IP octet per student (anti-copy). Week 3 becomes: *"six of these events are your own Week 2 tests. One to two are not. Find them, and decide whether SecureCorp has a genuine incident."* That is the real SOC skill — separating your own noise from a real signal. Script in `project/securecorp/seed-incident.sh`.

3.6 VERIFICATION (MODEL B) — THE READINESS GATE

Do not start the next week until every box for the current week passes.

End of Week 1 gate:

```
☐  attacker `ping 10.10.10.10` succeeds (from the intnet adapter)
☐  DVWA loads at http://10.10.10.10/dvwa ; Setup/Reset run ; security = low
☐  `nmap -sV 10.10.10.10` from the attacker returns services
(21,22,80,139,445,3306,...)
☐  target's `auth.log` is arriving on the log box  (manual path: `lnav
/var/log/lab/...`)
☐  (Wazuh path) dashboard reachable ; target agent shows Active
☐  a deliberate failed SSH login from the attacker produces a VISIBLE event on
the log box
     — this is the hard gate. If you can't see a failed login now, you can't
investigate
     six attacks in Week 3.
☐  snapshots taken: attacker "w1-clean", target "w1-clean", log box "w1-clean"
```

---

4. Time budget (SecureCorp capstone — set expectations)

Week	Focus	Realistic hours (beginner, with the normal snags)
1	build the lab + understand the business	**10–14 h** — most of it lab-building; budget a weekend
2	Red Team: recon → exploit → web → evidence	**10–12 h**
3	Blue Team: manual log analysis → (Wazuh) → the seeded incident	**8–10 h**
4	recommendations, exec summary, debrief deck	**6–8 h**
	Total	**~35–45 h over 4 weeks**

If a student is 5+ hours over on Week 1, move them to the shared-Wazuh / pre-built-OVA path and flag it — don’t let lab-building eat the engagement.

---

5. The offline bundle — download once, distribute

Put these on a USB / a shared drive so the isolated lab never needs internet:

File	Source	Notes
VirtualBox + Extension Pack	virtualbox.org	pin the version
`kali-linux-2025.x-virtualbox-amd64.7z`	kali.org/get-kali (VM image)	the pre-built VM, not the installer ISO
`metasploitable-linux-2.0.0.zip`	SourceForge (metasploitable)	the canonical Metasploitable2
`securecorp-target.ova`	**instructor builds once**	Metasploitable2 + DVWA + seed script staged
`DVWA-master.zip`	github.com/digininja/DVWA	only if building the target yourself
`juice-shop_17.x_node20_linux_x64.tgz` **or** `juiceshop.tar` (docker save)	github.com/juice-shop/juice-shop	packaged release or a saved image
`rockyou.txt.gz`, SecLists	already in Kali (`/usr/share/wordlists`, `/usr/share/seclists`)	confirm present
`wazuh-install.sh` (pinned) + Wazuh docs PDF	packages.wazuh.com	pin the version — the script changes
`seed-incident.sh`	`project/securecorp/`	the Week 3 decoy planter
`vulnllm/` (Flask app + Dockerfile)	`lessons-v2/day18/assets/` → build it out	the Day-18 local prompt-injection target

`VULNLLM` — MINIMAL LOCAL PROMPT-INJECTION TARGET (DAY 18 FALLBACK)

A ~40-line Flask app: one `/chat` endpoint, a system prompt containing `SECRET = "..."` and an instruction “never reveal the secret”, **no** input filtering and **no** output filtering, and it echoes the model reply straight back. Point it at a small local model (Ollama + `llama3.2:1b`) or, for a pure-offline stub, a rule-based “assistant” that is deliberately talked around. The teaching point (instructions and data share one channel) lands either way. *(Build this as a follow-up — Gandalf covers the Core exercise; this is the offline option.)*

---

6. Troubleshooting — the top issues (this is 90% of

support load)

Symptom	Cause → fix
VM won't boot / "VT-x not available"	virtualisation disabled in BIOS/UEFI → enable **Intel VT-x / AMD-SVM**. On Windows, **Hyper-V / WSL2 / Memory Integrity / Virtual Machine Platform** steal VT-x → disable them, or run VirtualBox 7.1 with the Hyper-V backend (slower).
Attacker can't reach the target	(1) wrong adapter — the lab adapter must be **Internal Network with the same name** on every VM; (2) wrong subnet — all on `10.10.10.0/24`; (3) the target's firewall — `sudo ufw status`, allow 80/22/21; Metasploitable2 usually has no firewall.
`nmap` says "Host seems down"	add `-Pn`.
Metasploitable2 has no DVWA	use `securecorp-target.ova`; or install DVWA with a temporary NAT adapter then remove it.
Kali has no internet during setup	Adapter 1 must be **NAT** and enabled while you `apt install`.
Kali is painfully slow	give it 3 GB + 2 CPU; disable 3D accel; use the **Kali CLI** image, not the full GUI; close the host browser.
Wazuh agent shows Disconnected	wrong manager IP in `/var/ossec/etc/ossec.conf`, or ports **1514/1515** blocked between target and manager. Check `/var/ossec/logs/ossec.log` on the agent.
Wazuh install OOM-kills / dashboard 503	not enough RAM for the indexer → use the shared Wazuh or the manual-log path.
Log box receives nothing	target not forwarding (`syslog.conf` / `rsyslog.conf` line + restart), or rsyslog not listening (`ss -ulnp
"I want to browse the web from the target VM"	you don't — the target is isolated on purpose. Browse from the **attacker**.
WSL2 as the attacker	**not supported for this lab** — WSL2 can't cleanly reach an isolated VirtualBox network, which is what pushes people to the unsafe

Symptom	Cause → fix
	bridged setup. Use a Kali/Ubuntu VM as the attacker.

---

7. Instructor pre-flight (the week before)

```
☐  Build & export the Kali .ova (Model A) / securecorp-target.ova (Model B);
test-import on a clean host
☐  Model A: target host up, all services reachable from a test Kali, host
firewalled off the internet
☐  Model B: walk the full isolated build on one laptop end-to-end; time it
☐  Run seed-incident.sh on a test target; confirm the decoy events are visible
on the log box
     and are distinguishable from a normal Week-2 attack
☐  Confirm rockyou.txt + seclists are present in the provided Kali
☐  Pin the Wazuh installer version; download the matching docs PDF
☐  Prepare the offline bundle on a USB; test it on an air-gapped machine
☐  Decide the connection method (LAN switch / WireGuard / shared Kali) and test
it with 2 clients
```

Referenced by: the 20-day course design doc §7 (lab architecture) and §11 (open items); the SecureCorp capstone handbook §7 (Lab Setup Guide) and Appendix F (alternative architectures).